The Python output of *shortest.py* looks like this:

```
Traceback (most recent call last):
  File "shortest.py", line 4, in <module>
    shortest()
  File "shortest.py", line 2, in shortest
    shortest()
  File "shortest.py", line 2, in shortest
    shortest()
  File "shortest.py", line 2, in shortest
    shortest()
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

The JavaScript output of *shortest.html* looks like this in the Google Chrome web browser (other browsers will have similar error messages):

```
Uncaught RangeError: Maximum call stack size exceeded
    at shortest (shortest.html:2)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
    at shortest (shortest.html:3)
```

This kind of bug is called a *stack overflow*. (This is where the popular website *https://stackoverflow.com* gets its name.) The constant function calls with no returns grows the call stack until it uses up all of the computer's memory allocated for the call stack. To prevent this, the Python and JavaScript interpreters crash the program after a certain limit of function calls that don't return.

This limit is called the *maximum recursion depth* or *maximum call stack size*. For Python, this is set to 1,000 function calls. For JavaScript, the maximum call stack size depends on the browser running the code but is generally at least 1,000 as well. Think of a stack overflow as happening when the call stack gets "too high" (that is, consumes too much computer memory), as in Figure 1-8.

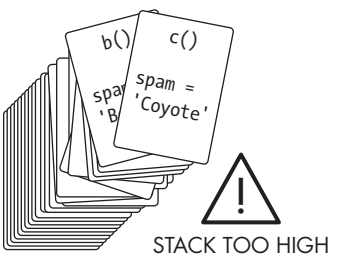

Figure 1-8: A stack overflow happens when the call stack becomes too high, with too many frame objects taking up the computer's memory.